D6 Continuous Integration

Primary: Samuel Knight, Emma Fuller
Secondary: Leo Durnion
Tertiary: Hamza Ahmad, Wasif Mustafa, Anexa Bijoy

D6a - Continuous Integration Approach

In order to use continuous integration in our project we made use of automated testing and building to allow us to perform quick and thorough tests each time a commit is made.

As a team, we also make sure that all changes are completed on branches that are short-lived so that the changes can be pushed back onto the main branch as soon as possible. By having the commits as small and frequent as possible, it ensures that the risk of major conflicts will be reduced. This will allow for a smoother development journey overall [1]. We should make sure to push to the remote at least once a day to ensure that the remote is as up-to-date as possible. The main branch will be the single source of truth to keep our project on track.

Every member of the team will ensure that the Kanban will also be kept up to date so that we will all be aware of what is happening in the other areas of development so they can work more seamlessly together even when developing different areas of the game. This is vital for a successful project especially in the short life cycles that we have for this project.

Each time a commit is pushed, the automated testing and build workflow will be run to ensure that the changes will not break the game in its current state. This ensures that the testing process is as quick and efficient as possible, this rapid feedback ensures we can stay on track for development [2]. After the build workflow completes, the compiled executable JAR artifact will be uploaded to allow us to manually test the contents of the PR before approving it. We'll achieve this using GitHub Actions workflows.

Additionally, we want our tests and build process to run on multiple environments: Linux, Windows, and Mac. We can do that using matrices in actions workflows. By doing this, we can ensure that the game compiles and passes all tests in all major environments without needing a member of our team to have a physical device with the OS installed.

We also want to ensure that code complies to the Google Java styleguide. To achieve this, we could enforce manual style checks for every PR. But we can automate it instead using the Checkstyle plugin [3]. We can integrate this into our workflows using the Gradle plugin, and run the checks for every PR using GitHub Actions again.

Finally, we want to generate coverage reports to show the coverage of our test reports. To do this, we'll use the JaCoCo plugin, and another automated workflow to report on the code coverage. We won't be aiming for a specific percentage coverage as this is often unrealistic and not strictly necessary to make the project stable enough for our purposes. Instead, this will be more of an indicative metric to show progress with the testing deliverable and whether tests still cover changes to implementation.

D6b - Continuous Integration Infrastructure

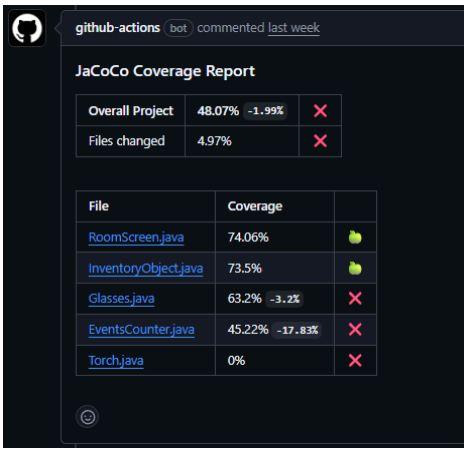
Tasks (represented as issues on a GH Project) which involved code changes had a pull request (PR) linked to them with a unique branch per task. These were set up so that when the PR was marked as ready for review, the tasks would automatically move to the “In Review” column, then when they were closed, the linked issue would also close and move to the “Done” column.

All code changes were made in this way, creating a branch with a pull request linked to an issue. We also required that PRs were reviewed by a member of the team independent of the person who opened the request. This meant that code was always looked at by a second pair of eyes to make sure that any simple mistakes were caught.

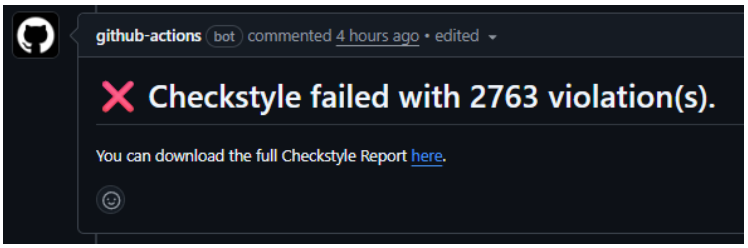
We also set up 3 workflows to perform key checks before a PR was approved to be merged into the master branch:

- **Java CI with Gradle** - This workflow automated the build process, including running all unit tests. If any test failed, the workflow would fail, indicating that the branch should not be merged until the part of the project that the test failed on was amended. This workflow also uploaded the built JAR file as an artifact, allowing reviewers to quickly run and verify the latest builds. This build process was also set up to use a matrix strategy, creating a build for the latest versions of Ubuntu, Windows, and macOS

- **Measure Coverage** - We installed the JaCoCo library to our development environment using Gradle. This created the `jacocoTestReport` task that could be run to measure the coverage of our tests. Utilising this task, we created a workflow that runs on PRs to trigger this task. Then, the report is uploaded as an artifact in case it’s needed, but usually we didn’t need to do this because we also used the `madrapps/jacoco-report` action to summarise coverage results in a PR comment.



- **Checkstyle** - Finally, we installed the Checkstyle plugin to analyse the style of our code and enforce consistent rules in line with Google’s style guide. This created the `checkstyleMain` task which could be run to generate a report of all of the style discrepancies. We then (similarly to JaCoCo) set up a workflow to run this task and report the findings in a comment on the PR.



Towards the end of the project, we had to go back and modify our matrix build to exclude macOS. This is because the cost of running actions on macOS is significantly higher than either Windows or Linux, so we ended up using our whole budget of Actions minutes, and didn’t have the budget to continue builds on macOS.

References

[1]M. Chukwube, “Why Continuous Integration Matters More Than Ever - DevOps.com,” *DevOps.com*, Aug. 08, 2025. https://devops.com/why-continuous-integration-matters-more-than-ever/ (accessed Nov. 28, 2025).
[2]M. Fowler, “Continuous Integration,” *martinfowler.com*, Jan. 18, 2024. https://martinfowler.com/articles/continuousIntegration.html
[3] Checkstyle, *checkstyle.sourceforge.io*, https://checkstyle.sourceforge.io/ (accessed Dec. 05, 2025)